قوافل Qawafel

By Meshari!

TECHNICAL ASSESSMENT · SUBMISSION REPORT

Graduate Software Engineer

helping a wholesaler get financed

CANDIDATE	DEADLINE	SCOPE
Meshari Applicant	**Saturday 24 October 2026** 11:59 PM, Riyadh time	**Tasks 1 to 5** Task 5 is a bonus

Topics covered

The Problem

Assumptions and Decisions

Task 1: Requirements
1.1 Users and user stories
1.2 Questions for the customer
1.3 What could go wrong with the data

Task 2: Technical Spec
2.1 Data model
2.2 API
2.3 Process flow

Task 3: AI Prototype
Link, how to run, where the sample data is

Task 4: Integration Judgment
4.1 Integration plan
4.2 What I'd say to the bank

Task 5: Automating the Request (Bonus)
5.1 What to automate first, and why
5.2 Where the data could come from instead of uploads
5.3 Where a human stays in the loop
5.4 How we'd know it's better

AI Tools Used

The Problem (in my own words)

An SME (small or medium-sized enterprise) wholesaler sells to retailers on credit, so its cash sits in unpaid invoices while it still has to pay its suppliers. It needs financing, such as a working-capital loan, to bridge that gap. But the bank cannot tell from the SME's own claims whether the business is real and able to repay.

Qawafel's job is to make those claims checkable: turn 12 months of invoices (Excel) and bank statements (PDF) into one verified package by matching what was invoiced against what was actually paid, then send it to the bank partner for a decision.

Assumptions and Decisions

The brief leaves several choices open on purpose, so I made a call on each one and wrote it down. Together they set the direction of the project:

a cash-flow loan for a food wholesaler, with the bank's credit reviewer as the prototype's main user.

ID	ASSUMPTION	WHY	IF THIS IS WRONG
A1	**Financing type** A working-capital loan or credit line, decided on the SME's overall cash flow.	The brief gives the bank 12 months of invoices and statements for the whole business, which is what cash-flow lending reads.	Invoice factoring (selling unpaid invoices to a financier for cash) would also need to confirm each open invoice and the buyer who owes it. The invoice-to-payment matching I build still applies.
A2	**Users** Task 1 covers three users: the SME owner, a Qawafel analyst who checks the package, and the bank's credit reviewer. The prototype is built for the credit reviewer.	The prototype is judged on whether it helps someone make a financing decision, and the bank makes that decision.	If the SME view matters more, the same data-quality flags become a "fix before you submit" screen inside the prototype.
A3	**Roles** Qawafel prepares and verifies the package. The bank partner makes the credit decision. Qawafel does not lend or approve.	The scenario says Qawafel sends the package to a bank partner for review.	If Qawafel also scored the risk, the package would carry its own recommendation next to the bank's decision. The core package stays the same, so the impact is small.
A4	**The wholesaler** A fictional Saudi food wholesaler. Amounts in SAR with 15% VAT, selling to retailers on assumed credit terms of 30 to 60 days.	Public information shows Qawafel working with food suppliers and retailers, so this is a realistic customer. The credit terms are my estimate. All data is made up, as the ground rules require.	Terms, currency and tax are settings, so the sample data can be regenerated with other values to match reality more closely.
A5	**Sample data** The 12 months of sample data is deliberately imperfect: duplicates, credit notes, late and partial payments, and mixed Arabic and English names. It was created with AI tools.	Clean data is not believable, and it would hide the risks listed in Task 1.	The checks still work on clean data. A clean set would just show less of what they catch.

All work in this report follows the assumptions and decisions above.

Task 1: Requirements

Three users, eight user stories, nine questions for the customer and ten data risks. The stories rest on assumptions A1 to A5 above.

1.1 Users and user stories

USER	WHO THEY ARE	WHAT THEY NEED	STORIES
SME owner	The wholesaler's owner, or a finance person working there, who applies for the loan.	To submit the files once, know what to fix, and see where the application stands.	S1, S2
Qawafel analyst	Qawafel staff who create and check each package before it goes to the bank.	To review only the unclear cases, and to send a package they can stand behind.	A1, A2, A3
Bank credit reviewer	Works at the bank partner and decides on the loan. The prototype is built for this user.	To judge the business in minutes and check any figure against its source.	B1, B2, B3

Not users. The retailers who owe the SME appear in the data but never use the system, and the bank's own systems are covered in Task 4.

Terms. A package is the verified set of invoices, statements, matches and flags that goes to the bank. A match links a bank deposit to the invoice it pays. A credit note reduces an earlier invoice, for example when goods are returned. A flag is a problem found in the data. It is either blocking (it must be resolved before sending) or a warning (it is shown to the reviewer). Accepted in principle means the bank agrees to lend, subject to its own final checks with the SME. It is not yet an approval. Approved means those checks are done and the bank confirms the financing.

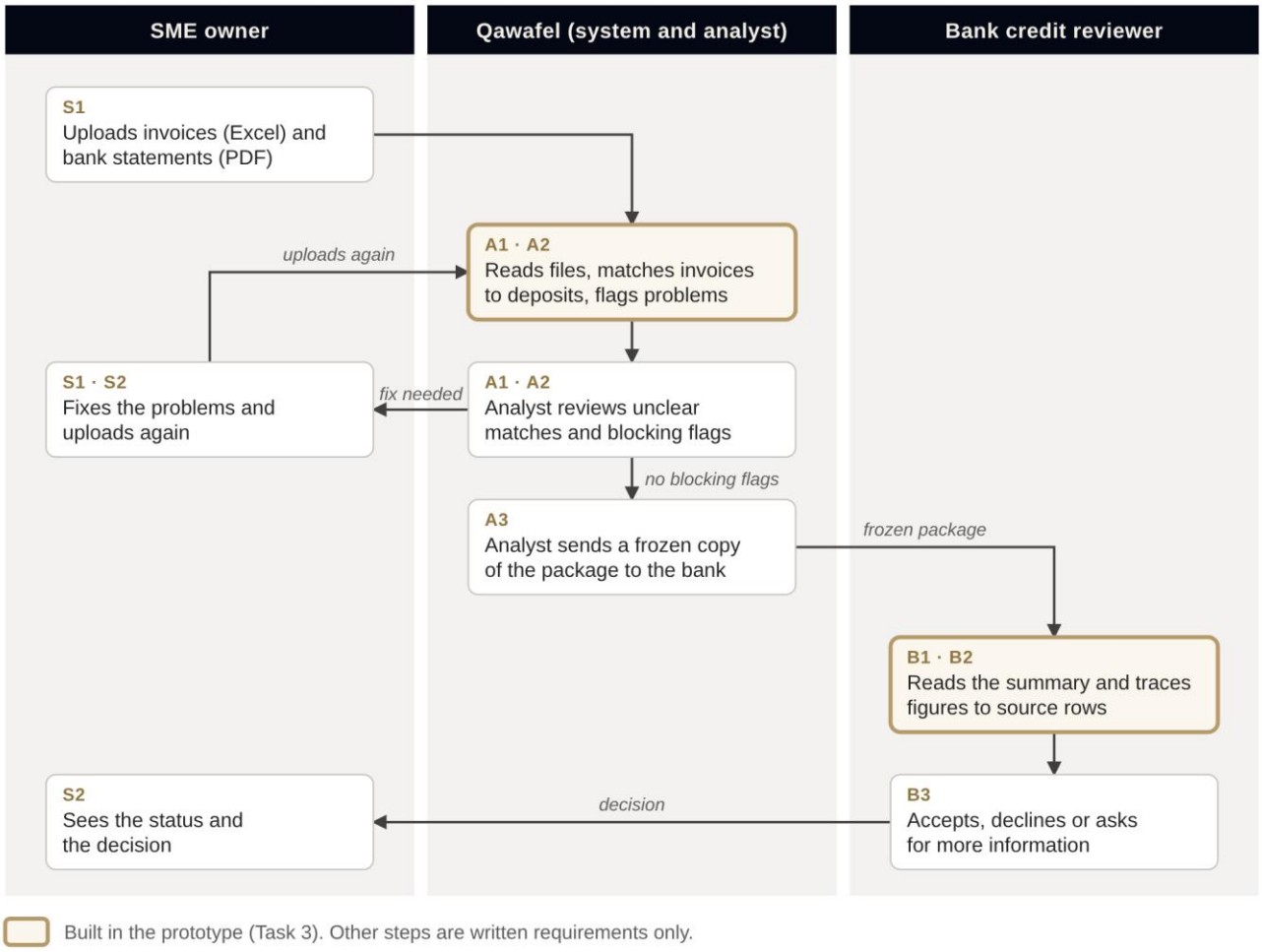


Figure 1. Who does what, from upload to decision. The story IDs match the table below.

ID	USER STORY	ACCEPTANCE CRITERIA	IN TASK 3?
SME OWNER			
S1	**As an** SME owner, **I want** to upload my invoices (Excel) and bank statements (PDF) and see any problems in plain language, **so that** I can fix them before the bank sees anything.	• Only .xlsx and .pdf files are accepted. Any other type is rejected with a message that names the accepted types. • After upload, I see what was read: the number of invoices, the statement period and the number of transactions. • Each problem names the file and the row or page, and says what is wrong in one plain sentence.	No
S2	**As an** SME owner, **I want** to see where my application stands, **so that** I do not have to chase Qawafel.	• Status is always one of: Draft, In review, Needs changes, Sent to bank, Accepted in principle, Approved, Declined. • Accepted in principle, Approved or Declined appears only after the bank's answer is confirmed, not on its first response (see Task 4). Until then the status stays Sent to bank. Accepted in principle is not an approval, and the screen says so. • Needs changes shows what I must fix, or what the bank asked for.	No

ID	USER STORY	ACCEPTANCE CRITERIA	IN TASK 3?
QAWAFEL ANALYST			
A1	**As a** Qawafel analyst, **I want** invoices matched to bank deposits automatically, **so that** I can check a package in minutes.	• Every invoice ends as paid, partly paid or unpaid, and every deposit as matched or unexplained. • Matches the system is unsure about are listed for my review. I do not have to check the rest. • I can change a match by hand. The system records who changed it and when.	Partly
A2	**As a** Qawafel analyst, **I want** the system to flag the data risks listed in part 1.3, **so that** bad data does not reach the bank.	• At a minimum it flags these problems whenever they occur: duplicate invoices, missing months, unreadable statement pages, statements whose balance does not add up, and gaps in invoice numbering. • Each flag is marked blocking or warning, with the reason in plain language. • A package cannot be sent while a blocking flag is open. I can resolve a flag with a short note, which is kept with the package.	Partly
A3	**As a** Qawafel analyst, **I want** to send a frozen copy of the package (a locked version that can no longer change) to the bank, **so that** we can prove exactly what the bank saw.	• Sending locks that version. A later change creates a new version and does not alter the one sent. • A log records who sent which version, to which bank, and when. • A package cannot be sent without the customer's recorded consent.	No
BANK CREDIT REVIEWER			
B1	**As a** bank credit reviewer, **I want** a one-page summary of the business (monthly revenue, share confirmed by bank deposits, share of the top 5 customers, average days to pay, flags), **so that** I can judge in minutes whether to look deeper.	• Shows monthly revenue for all 12 months, with the trend. • Shows the share of invoiced revenue matched to bank deposits, the share of revenue from the top 5 customers, and the average days to pay with its trend. • Lists every flag with its reason in plain language.	Yes
B2	**As a** bank credit reviewer, **I want** to click any figure and see the invoices and statement lines behind it, **so that** I do not have to take Qawafel's word for it.	• Every number on the summary opens the rows it was calculated from. • Each source row shows its match status.	Yes
B3	**As a** bank credit reviewer, **I want** to accept in principle (agree to lend, subject to the bank's own final checks), decline or ask for more information, with a reason, **so that** Qawafel and the SME get a clear answer.	• Declining or asking for more information requires a written reason. • The decision is saved with my name and a timestamp. • The SME and Qawafel see the outcome as Accepted in principle, Declined or Needs changes. Final approval comes later, after the bank's own checks with the SME.	No

1.2 Questions for the customer

In real work I would ask these questions first and write the stories afterwards. Here the stories came first, so these are the questions that would confirm them. The answers could change the stories, and the last column shows which ones.

#	QUESTION	WHY IT MATTERS	COULD CHANGE
INVOICES			
Q1	Where do the invoices come from (an accounting system, e-invoicing, or hand-made Excel), and can you send the same export every month?	Decides whether we parse one stable format or many.	S1, A2
Q2	Is it one row per invoice or per item, and one file for 12 months or one per month?	Sets the data model and how we detect duplicates.	S1, Task 2
Q3	How do you record returns, discounts and corrections?	Without credit notes, revenue is overstated.	A2, B1
MONEY COMING IN			
Q4	Do all customers pay into the account you are submitting, or some by cash, cheque or another account?	If not, invoices will not match deposits, and honest revenue will look unsupported.	A1, B1
Q5	Does a transfer mention the invoice number or only the customer's name?	Decides how many matches need the analyst.	A1
Q6	Which banks and how many accounts, is the statement a bank PDF or a scan, and does it also carry personal money, loans or transfers between your own accounts?	Scans are less accurate, layouts differ per bank, and non-sales deposits would be counted as revenue.	S1, A2
CUSTOMERS AND TERMS			
Q7	What credit terms do customers get, and who are your biggest accounts?	Sets the late-payment checks and the concentration risk.	B1
THE REQUEST			
Q8	How much do you need, for what, and how quickly?	Sets what "useful" means for the reviewer.	B1, B3
Q9	Do you agree to Qawafel sharing your data with the bank, and which bank?	Consent is needed before anything is sent.	A3

1.3 What could go wrong with the data

The first three risks are about reading the files: the data may be hard to read or to interpret correctly. The other seven read fine but can still mislead the bank. I rate the impact High when a risk could inflate revenue, because revenue is the first number a bank looks at. The same ten risks guided the sample data in Task 3, so it contains a realistic example of each (assumption A5).

#	RISK	WHAT IT LOOKS LIKE	WHY IT MATTERS	HOW WE CATCH IT	IMPACT
READING THE FILES					
1	**Mixed Arabic and English**	The same customer appears as Al Noor Co. in one file and شركة النور in another. Digits may be Arabic-Indic (١٢٣).	One customer is counted as two, so concentration looks lower and matching fails.	Normalise digits and spelling, match customers on the cleaned name, and keep the original text.	Medium
2	**Unusual numbers and dates**	Numbers stored as text, commas as decimals, brackets for negatives, VAT on some rows only, line totals that do not add up, Hijri or swapped day and month.	Revenue and days to pay come out wrong.	Convert to numbers and standard dates (year-month-day), check line totals against invoice totals, and flag mixed VAT treatment.	Medium
3	**Missing fields and incomplete files**	No invoice number or date, a missing month, a scanned or cut-off PDF page, invoice and statement periods that do not line up.	A gap reads as hidden revenue or a pause in the business.	Required-field check, month-by-month coverage check, and a flag on pages with no readable text.	Medium
DATA THAT READS FINE BUT MISLEADS					
4	**Duplicate invoices**	The same invoice exported twice, or edited and re-numbered.	Revenue is inflated.	Flag the same invoice number, or the same customer, date and amount.	**High**
5	**Credit notes, returns and cancellations not netted off**	A return stays in as a sale, or a credit note has no invoice to attach to.	Revenue is inflated.	Net each credit note against the invoice it refers to, and flag any with no match.	**High**
6	**Payments that do not match 1 to 1**	One deposit pays three invoices, partial payments, overpayments.	False “unpaid” flags, so the analyst has to review.	Allow one-to-many and partial matches, and send low-confidence matches to the analyst.	Medium
7	**Non-sales money in the statement**	Owner deposits, loans, transfers between own accounts, refunds.	Cash received looks higher than sales.	Tag these by rules and keywords, and leave them out of revenue received.	**High**
8	**Invoices left out**	Gaps in the invoice numbering suggest only the good invoices were sent.	A flattering picture, with late payers hidden.	Check the numbering for gaps and show them.	Medium
9	**Circular or related-party money**	Money goes out to a “customer” and comes back as a payment, or the customer is the owner’s other company.	Fake sales.	Flag money that leaves and returns in the same amount within days, and customers that are also payees.	**High**
10	**Edited or inconsistent statements**	Opening balance plus deposits minus withdrawals does not equal the closing balance.	The statement cannot be trusted.	Recalculate the running balance and flag any difference.	Medium

Task 2: Technical Spec

This task covers the data model, the API and the end-to-end process flow. The model covers the whole product. The prototype in Task 3 uses only the entities outlined in gold.

2.1 Data model

In plain words: a business makes an application. The application holds two files, the invoices and the bank statement. The system reads them into invoices and bank transactions, links each deposit to the invoices it pays, and raises a flag for every problem. A frozen copy of the result is sent to the bank, and the bank's answer is stored with it.

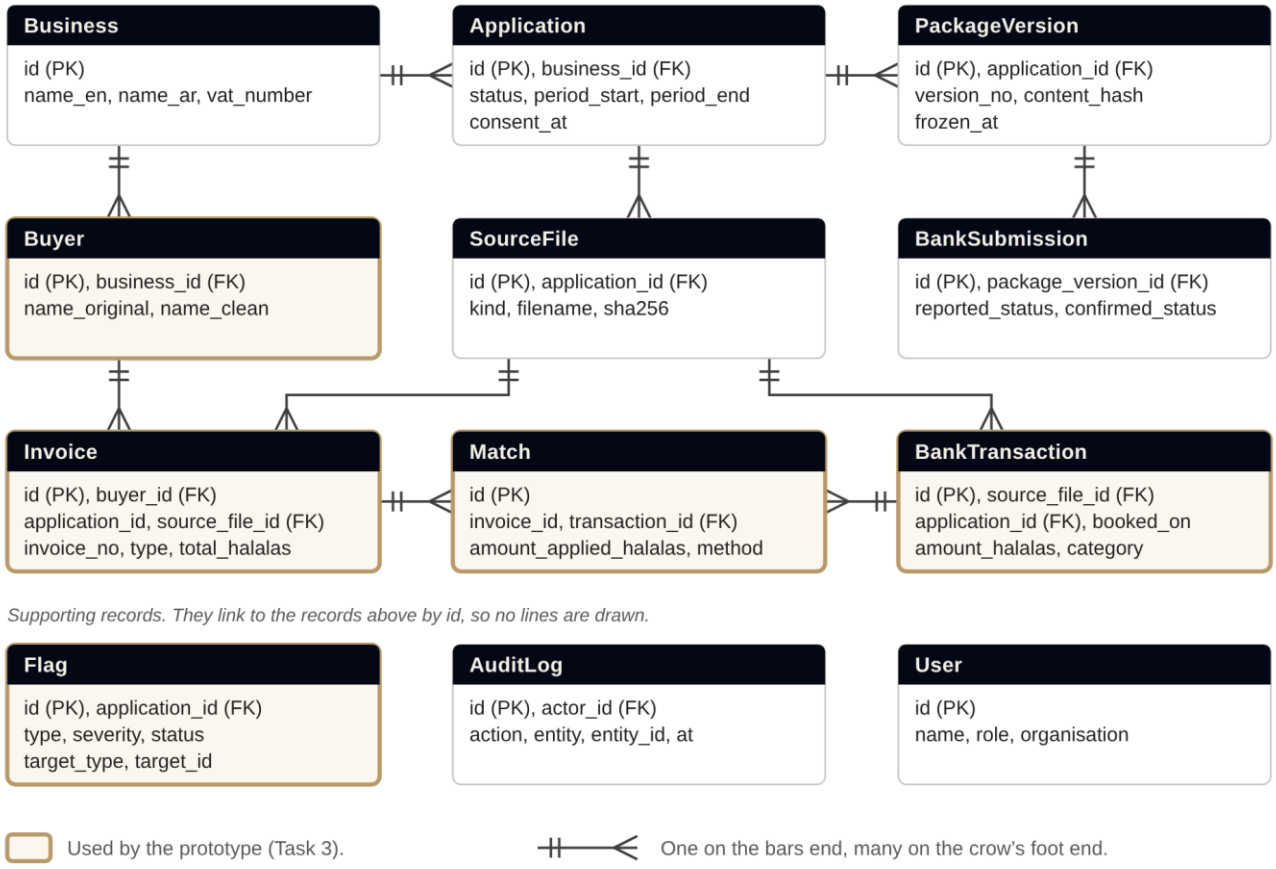


Figure 2. Data model. Invoice, BankTransaction and Flag also carry application_id, so a whole application can be read without joining through files.

Entities and fields

Types. text is words kept as written; date is a calendar date; timestamp is a date and time in UTC; int is a whole number; bigint is a much larger whole number, used for money; decimal is a number with digits after the point; bool is yes or no; json is structured data kept as one value; uuid is a unique id; enum is a fixed list of allowed values; sha256 is a fingerprint of a file's contents (the same content always gives the same fingerprint). PK marks a row's own id (primary key) and FK a link to a row in another table (foreign key). Money is always a whole number of halalas (100 halalas = 1 SAR), so 1,150.00 SAR is stored as 115000.

ENTITY	FIELDS	RULES
Business	id (uuid, PK), name_en (text), name_ar (text), vat_number (text), cr_number (text), created_at (timestamp)	vat_number is unique. Names are kept in both languages, as entered.
Buyer	id (uuid, PK), business_id (FK), name_original (text), name_clean (text)	name_clean is the normalised name (digits, spelling, case) and is unique per business. The original is always kept (data risk 1).
Application	id (uuid, PK), business_id (FK), status (enum: draft, in_review, needs_changes, sent_to_bank, accepted_in_principle, approved, declined), period_start (date), period_end (date), consent_at (timestamp, null until given), consent_by (FK User), created_at (timestamp)	Status moves only through allowed transitions. It cannot become sent_to_bank without consent_at or while a blocking flag is open. accepted_in_principle, approved and declined are set only from a confirmed bank answer.
SourceFile	id (uuid, PK), application_id (FK), kind (enum: invoices_xlsx, statement_pdf), filename (text), sha256 (text), byte_size (bigint), storage_key (text), uploaded_by (FK User), uploaded_at (timestamp)	Unique on (application_id, sha256), so the same file cannot be added twice. The file itself is kept in a file store (object storage), not in the database.
Invoice	id (uuid, PK), application_id (FK), source_file_id (FK), source_row (int), buyer_id (FK), invoice_no (text), type (enum: invoice, credit_note), credit_note_of (FK Invoice, null), issue_date (date), due_date (date, null), net_halalas (bigint), vat_halalas (bigint), total_halalas (bigint), currency (text, default SAR), raw (json, the original row)	Credit notes are stored with negative totals, so sums net off. total = net + vat is checked on load, and a mismatch raises a flag. invoice_no is not unique: duplicates are stored and flagged.
BankTransaction	id (uuid, PK), application_id (FK), source_file_id (FK), source_page (int), booked_on (date), description (text, original), counterparty (text), amount_halalas (bigint, positive in and negative out), balance_after_halalas (bigint, null), category (enum: sales_receipt, loan, own_transfer, refund, other, unclassified), raw_text (text)	category is set by rules, and only sales_receipt counts as revenue received. balance_after is compared with the running total to catch edited statements (data risk 10).
Match	id (uuid, PK), invoice_id (FK), transaction_id (FK), amount_applied_halalas (bigint), confidence (decimal, 0 to 1), method (enum: auto, manual), needs_review (bool), created_by (FK User, null for the system), created_at (timestamp), superseded_at (timestamp, null)	Amounts applied to one transaction cannot exceed it. More than an invoice's total raises an overpayment flag. A manual change closes the old match (superseded_at) and adds a new one, so history is kept.
Flag	id (uuid, PK), application_id (FK), type (enum, see 1.3), severity (enum: blocking, warning), target_type (text), target_id (uuid), message (text), status (enum: open, resolved), resolved_by (FK User, null), resolved_note (text, null), created_at (timestamp), resolved_at (timestamp, null)	target_type and target_id point to the invoice, transaction or file concerned. An open blocking flag stops sending. Resolving needs a note.
PackageVersion	id (uuid, PK), application_id (FK), version_no (int), summary (json, the figures shown to the reviewer), content_hash (text), snapshot_key (text), frozen_by (FK User), frozen_at (timestamp)	Never updated or deleted. Unique on (application_id, version_no). content_hash is the sha256 of the frozen snapshot.
BankSubmission	id (uuid, PK), package_version_id (FK), bank (text), idempotency_key (uuid), sent_by (FK User), sent_at (timestamp), reported_status (text, as the bank said it), reported_at (timestamp), confirmed_status (enum: pending, accepted_in_principle, approved, declined, more_info_requested), confirmed_at (timestamp, null)	idempotency_key is unique, so a retry cannot send twice. Application status follows confirmed_status only, never reported_status (Task 4).

ENTITY	FIELDS	RULES
AuditLog	id (uuid, PK), actor_id (FK User), action (text), entity (text), entity_id (uuid), before (json), after (json), at (timestamp)	Append only. Written for every status change, manual match, flag resolution and send.
User	id (uuid, PK), name (text), role (enum: sme_owner, analyst, bank_reviewer), organisation (text), email (text)	email is unique. Role decides what each person can see and do.

Design decisions

#	DECISION	WHY
D1	Money is stored as whole halalas (bigint), never as decimals.	Rounding errors in a credit file destroy trust, and totals must add up exactly.
D2	Every parsed row keeps its file, its row or page, and its original text.	This lets the reviewer trace any figure to its source (B2), and lets us audit.
D3	Bad data is stored and flagged, not rejected.	Duplicates and odd rows are what the reviewer needs to see. Rejecting them would hide the risk.
D4	Match is its own table, with an amount applied.	One deposit paying three invoices, or one invoice paid in parts, becomes plain rows and not a special case.
D5	Flags are data: type, severity, status and a resolution note.	Supports A2. A blocking flag stops sending, and every resolution is recorded.
D6	A package version is immutable (it can never be changed) and carries a content hash (a fingerprint of its contents, so any change would show).	It proves exactly what the bank saw (A3).
D7	A bank answer is stored twice: reported and confirmed.	The bank's API is wrong 3% of the time (Task 4), so we act only on a confirmed status.
D8	Credit notes are Invoice rows with a type and a link to the original invoice.	One table to sum, and each credit note stays tied to the invoice it corrects.

2.2 API

The API follows one application from upload to the bank's answer. Each endpoint serves a story from Task 1, and the last column shows which.

Conventions

TOPIC	RULE
Style	REST over HTTPS with JSON bodies, all paths under /v1. Ids are uuids and dates are written year-month-day, for example 2026-10-20.
Sign-in and roles	A bearer token (a sign-in code sent with every request) carries the user and the role (Owner, Analyst, Reviewer). The system itself acts as a fourth role, System. A role that calls an endpoint it is not listed for gets 403.

TOPIC	RULE
Who sees what	An Owner sees only their own business. A Reviewer sees only frozen package versions sent to their bank, never live rows, so what the bank reads is exactly what was sent.
Money	Whole halalas as integers, matching the data model. 1,150.00 SAR is 115000.
Lists	Paged with limit and cursor (a marker for where the next page starts). Filters are query parameters, for example ?month=2026-03&needs_review=true.
Slow work	Parsing a file takes time, so the upload returns 202 (accepted, still working) with a file id, and the client checks that file until it is ready.
Safe retries	Sending to the bank requires an Idempotency-Key header (a unique code on the request). The same key returns the first result and never sends twice.
Errors	One shape for all: an HTTP code plus {"error": {"code", "message", "details"}}. The message is plain language. Codes used: 400 bad input, 401 not signed in, 403 wrong role, 404 not found, 409 rule broken (for example an open blocking flag), 415 wrong file type.

Endpoints

#	ENDPOINT	WHO	WHAT IT DOES	STORY
	UPLOAD AND TRACK			
1	POST `/applications`	Owner	Create an application for a business and a 12-month period. It starts as draft.	S2
2	POST `/applications/{id}/files`	Owner	Upload an .xlsx (invoices) or a .pdf (statement). Any other type gets 415 and a message naming the accepted types. Returns 202 and a file id.	S1
3	GET `/applications/{id}/files/{fileId}`	Owner, Analyst	What was read (invoice count, statement period, transaction count) and every problem, with its file and row or page.	S1
4	GET `/applications/{id}`	Owner, Analyst	Status, counts, open flags, and what to fix or what the bank asked for.	S2
5	POST `/applications/{id}/consent`	Owner	Record who agreed, when, and to which bank.	A3
	REVIEW			
6	GET `/applications/{id}/invoices` GET `/applications/{id}/transactions`	Analyst	Paged lists with filters (month, buyer, match status, flag). Every row carries its source file and row or page.	A1
7	GET `/applications/{id}/matches`	Analyst	Matches with their confidence. ?needs_review=true lists only the uncertain ones.	A1
8	POST `/matches`	Analyst	Create a match by hand: invoice, transaction, amount applied and a reason.	A1

#	ENDPOINT	WHO	WHAT IT DOES	STORY
9	POST `/matches/{id}/replace`	Analyst	Change a match. The old one is closed, a new one is added, and the change is logged with who and when.	A1
10	GET `/applications/{id}/flags`	Owner, Analyst	Flags with severity and a plain-language reason.	S1, A2
11	POST `/flags/{id}/resolve`	Analyst	Resolve a flag. A note is required and is kept with the package.	A2
12	POST `/applications/{id}/transitions`	Owner, Analyst	Move the status, for example to needs_changes with a message. Only the moves in the table below are allowed.	S2
PACKAGE AND SEND				
13	POST `/applications/{id}/package-versions`	Analyst	Freeze a version: compute the summary and the content hash. 409 while a blocking flag is open.	A2, A3
14	POST `/package-versions/{id}/submissions`	Analyst	Send a frozen version to a bank. Needs an Idempotency-Key header. 409 without consent. Logs sender, bank and time.	A3
15	POST `/submissions/{id}/confirm`	System	Store confirmed_status after a second check of the bank's answer and, for accepted, declined and final approval, the bank's written confirmation (Task 4). Only this changes the application status to accepted_in_principle, approved or declined.	S2
16	GET `/applications/{id}/audit`	Analyst	The audit trail: who did what, and when.	A1, A3
BANK REVIEWER (FROZEN VERSIONS ONLY)				
17	GET `/package-versions/{id}/summary`	Reviewer	The one-page summary: monthly revenue, matched share, top 5 customers, days to pay, flags.	B1
18	GET `/package-versions/{id}/figures/{key}`	Reviewer	The invoices and statement lines behind one figure, each with its match status.	B2
19	POST `/submissions/{id}/decision`	Reviewer	Accept in principle, decline or ask for more information. A reason is required for the last two. After the bank's own checks with the SME, the same call records final approval. Saved with name and time as reported_status.	B3

Status moves

The Application status changes only through these moves. Any other move returns 409.

MOVE	DONE BY	CONDITION
draft → in_review	Owner	At least one invoice file and one statement file were read.
in_review → needs_changes	Analyst	A message saying what to fix is required.
needs_changes → in_review	Owner	After a new upload or a reply to the message.
in_review → sent_to_bank	System	Happens when a frozen version is sent: consent is recorded and no blocking flag is open.
sent_to_bank → accepted_in_principle	System	Only when confirmed_status says so. The bank's first response alone never changes the status.
accepted_in_principle → approved	System	When the bank confirms final approval, after its own checks with the SME. What those checks are (a call, documents or a visit) depends on the bank.
sent_to_bank or accepted_in_principle → declined	System	Only when confirmed_status says so. The SME is told after confirmation, never before.
sent_to_bank → needs_changes	System	When the confirmed answer is "more information requested". The bank's question is shown to the Owner.

Examples

Freezing is refused while a blocking flag is open. Ids are shortened here.

```
POST /v1/applications/8e1c…/package-versions
409 Conflict
{
  "error": {
    "code": "blocking_flag_open",
    "message": "2 blocking flags are open. Resolve them before freezing a version.",
    "details": { "flags": ["f31a…", "f77b…"] }
  }
}
```

The reviewer's summary (B1) is read from the frozen version, so it cannot change after sending.

```
GET /v1/package-versions/c20d…/summary
200 OK
{
  "version_no": 2,  "content_hash": "9f3a…",
  "period": { "from": "2025-10-01", "to": "2026-09-30" },
  "monthly_revenue_halalas": [
    { "month": "2025-10", "value": 18450000 },
    { "month": "2025-11", "value": 17920000 }, … ],
  "matched_share": 0.87,  "top5_share": 0.41,  "avg_days_to_pay": 34,
  "flags": [ { "type": "duplicate_invoice", "severity": "warning",
               "message": "Invoice INV-1042 appears twice (rows 88 and 131)." } ],
  "figures": { "matched_share": "/v1/package-versions/c20d…/figures/matched_share" }
}
```

2.3 End-to-end process flow

Who does what, and where the process loops back: on a blocking flag, and on a bank answer that does not pass the second check.

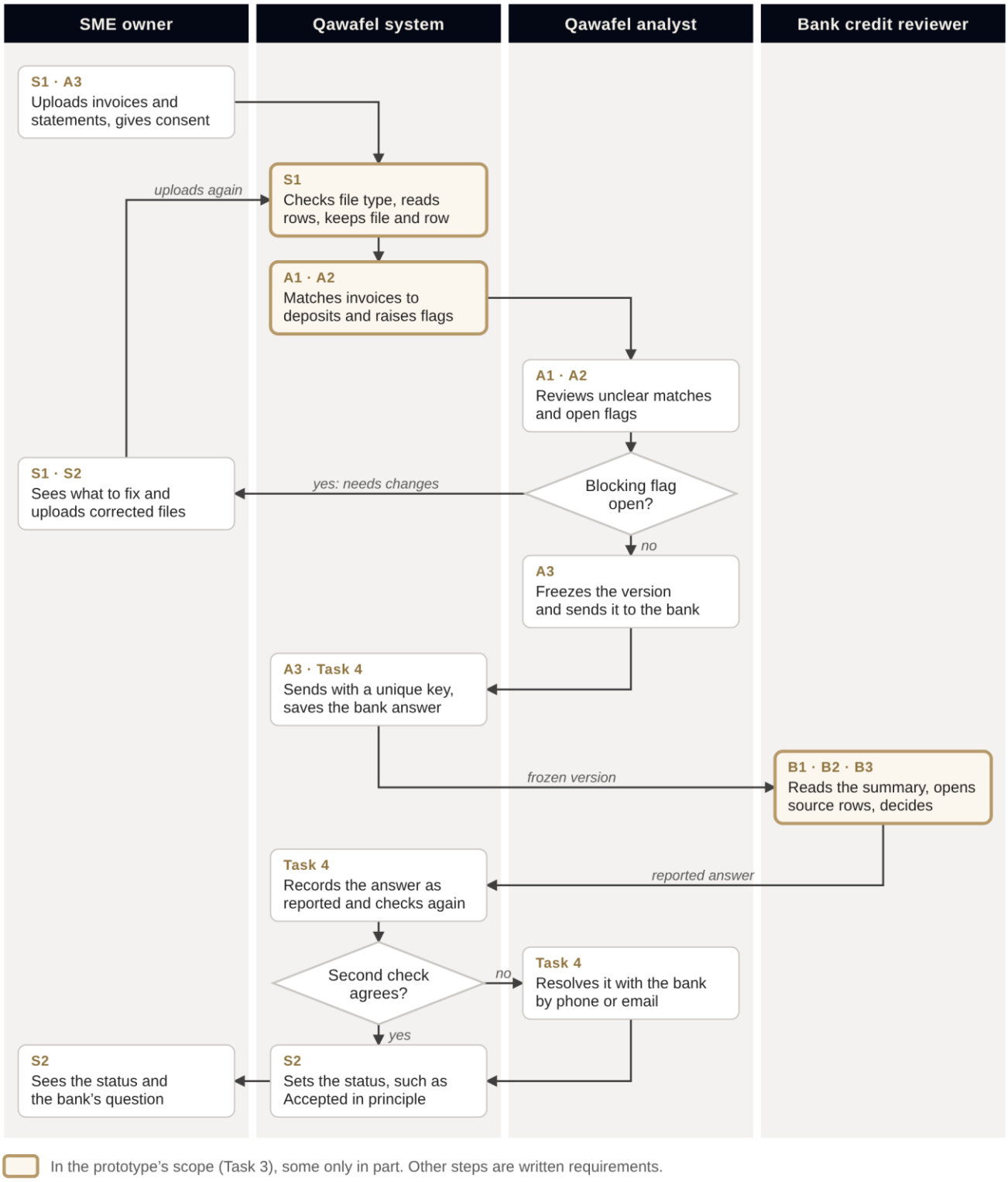


Figure 3. End-to-end flow, from upload to the bank's confirmed answer. The tags show the Task 1 stories each step serves.

Task 3: AI Prototype

A working prototype for the bank's credit reviewer. It answers one question: can this business repay a working-capital loan, and what in the files should a person check before the bank sees them?

Code and README: https://github.com/MeshMoh506/Qawafel_UseCase_ByMeshari

Live link: https://qawafel-usecase-bymeshari.onrender.com

ABOUT	ANSWER
What it does	Reads the invoice file and the bank statement, matches each deposit to the invoices it pays, raises issues, and shows a one-page review for the bank's credit reviewer. Every figure opens to the invoices and bank lines behind it.
What the reviewer sees	SAR 18.2M of genuine revenue, 85% matched to bank deposits, 93% of invoices due collected, 35 days to pay, and the top five customers at 42% of revenue. Five blocking issues must be resolved with a written note before the package counts as ready.
How to run it	To see it, open the live link above: nothing to install. To run it on your own computer you need Python and Node: build the React app, start the API, and open localhost:8000. The README has the commands and a Docker option. On free hosting the live link can take up to a minute to load the first time.
Where the sample data is	In data/sample in the repo: 862 invoice rows for 29 customers, a 28-page bank statement in Arabic and English, an answer key, and the seeded generator that made them. The business, bank and customers are made up.
Assumptions	In the README, in two groups: how the sample data was built, and how the prototype reads it. For example, a missing due date means 30 days, and revenue excludes VAT.
How well it works	On the sample data, 721 matches were made and 99.6% were right. 96.9% of the correct matches on readable pages were found. Every kind of planted problem was found, but one of four re-issued invoices was missed.
What is not built	File upload, editing a match by hand, freezing and sending a version, and reading the scanned page. These are written as requirements in Task 2. The prototype reports the unreadable page instead of guessing.

Task 4: Integration Judgment

The bank's API gives a wrong status about 3% of the time, roughly one answer in 33, and the plan has to allow for that rate. There is also no sandbox (a safe test copy of the bank's system) to try it on. So the plan rests on one rule: a bank answer is a claim to check, not a fact to act on.

4.1 Integration plan

What a wrong answer costs

The bank can say three things: accept in principle (agree to lend, subject to its own final checks), ask for more information, or decline. Each one fails differently, so each one gets different handling. The table shows the cost of a wrong answer and what we do about it.

BANK SAYS	IF IT IS WRONG	COST	WHAT WE DO
Accepted in principle	The SME plans around money that will not come.	**High**	Show "Accepted in principle", never "Approved", and only after the answer is confirmed. The bank's own checks with the SME come before final approval, a second layer. It is on the bank's side, so we do not rely on it alone.
Needs more information	The SME is asked for documents it does not need to send.	**Low**	Confirm first, then pass on the bank's exact words. A wrong one costs a day, not a deal.
Declined	A good application is lost, and no later step catches it.	**High**	Hold it until confirmed. If the two checks disagree, the analyst calls the bank before the SME hears anything.
No answer, or the same answer twice	The status stays on "Sent to bank", or an update is applied twice.	**Medium**	Duplicates are ignored. After the reply time agreed with the bank, the analyst asks the bank.

Rules built into the system

#	RULE	WHY
R1	**Two statuses.** We store what the bank said (reported_status) and what we confirmed (confirmed_status). Only the confirmed one changes what the SME and the analyst see.	A wrong answer is kept for inspection and does no harm.
R2	**Confirm with a second read.** After an answer we ask the bank for the status again after a short delay. If the two agree, it is confirmed. If they differ, the analyst resolves it with the bank.	Catches random errors, such as an old or wrong value.
R3	**Confirm in writing.** For accepted, declined and final approval we also ask for the bank's reference or letter, or compare with its daily report.	A second read from the same API can repeat the same mistake. Only an independent source catches that.
R4	**Allowed moves only.** Any move outside the status moves in 2.2 is stopped and shown to the analyst, for example "Declined" followed by "Sent to bank".	A wrong status often looks impossible, and checking costs nothing.

#	RULE	WHY
R5	**Two-step approval.** A first accept shows as “Accepted in principle”. Only the bank’s final approval, after its own contact, documents or visit, shows as “Approved”.	The SME never hears “approved” on the strength of one answer, and the screen says what happens next.
R6	**Send once.** Each send carries an Idempotency-Key (a unique code, so a repeat is recognised), and the package version has a content hash. After a timeout we ask for the status before sending again. A changed file is a new version with a new key.	A retry cannot create a duplicate or change what the bank is reviewing.
R7	**No action from a status alone.** Nothing leaves Qawafel, such as a payment or a contract, because a status changed.	The worst wrong answer cannot cause real damage.

How we test without a sandbox

A sandbox is a safe copy of the bank’s system for practice. We do not have one, so we practise on a pretend bank first, then on test applications, then on a few real ones with a person watching.

STAGE	WHAT WE DO	WE MOVE ON WHEN
1. Stand-in bank	We build a small fake bank from the bank’s documentation. It is wrong 3% of the time on purpose, and is sometimes slow, down or repeats itself. Our tests run against it on every build.	Every case ends in the right status or in front of a person, and no wrong status reaches an SME.
2. Shadow mode	We try the integration without acting on any answer. If the bank agrees, we send made-up applications marked as tests. If not, we only read statuses and compare them with what the bank’s staff tell us. Nothing is shown to an SME.	Our confirmed answers match the bank’s staff on every test case.
3. Supervised pilot	The first real applications, a small number. The analyst confirms every status with the bank contact before the SME sees it. One switch stops all sending.	An agreed number of applications (say 20) with no wrong status shown to an SME.
4. Normal running	The analyst steps in only when R2 to R4 raise something. A daily job compares our records with the bank’s export.	Continues while the measures under this table stay in range.

What we watch. How often the second read differs from the first (about 3% is expected, and more means something changed), wrong statuses shown to an SME (the target is zero), time from the bank’s answer to a confirmed status, and applications waiting for the analyst.

Drawback of this plan. The analyst is in the loop at stages 3 and 4, which costs time. If the bank’s daily report proves reliable, most answers can be confirmed automatically and the analyst asked only about differences.

4.2 What I’d say to the bank

A short call in this order: what we are building, what “wrong” means, then the few things that make it safe. I would send the questions in advance.

Opening

“Thank you for making the time. We send financing packages for wholesalers to you, and we show the customer what happens next. We understand your status API is wrong about 3% of the time and that there is no test environment. We do not want a wrong status to reach a customer, so we check every answer before we act on it. I would like to confirm we understand the problem, ask a few questions, and agree a few things with you.”

Questions

#	QUESTION	WHY WE ASK
Q1	When a status is wrong, what is wrong: an old status, a different one, or none? Does it correct itself, and how soon?	Tells us how long to wait before the second read.
Q2	Is the 3% spread evenly, or does it happen at certain steps, such as just after a change? How was it measured?	Lets us put the check where the errors are.
Q3	Does each answer carry a last-updated time or a version number? Can we see the status history?	An old answer can be spotted without a second call.
Q4	Do you push updates, or do we ask? What are the rate limits? Can the same update arrive twice?	Decides how we receive and de-duplicate.
Q5	Does the send call accept our own reference or an idempotency key? What should we do after a timeout?	Prevents duplicates (R6).
Q6	After you accept, what happens next: who contacts the SME, which documents, is there a visit, how long does it take, and how do we hear the final result?	We show the SME the right next step and design the final confirmation (R5).
Q7	Can we test before the first real application, with test applications or a test key?	There is no sandbox.
Q8	Who do we contact when we think a status is wrong, and how fast do you reply?	Needed to resolve a disagreement (R2).

What I would ask them to agree to

#	WE WOULD ASK YOU TO AGREE TO	WITHOUT IT
A1	One reference per application, in every answer.	We would match answers to applications by guessing.
A2	Written confirmation for accepted, declined and final approval.	We cannot catch a mistake that the API repeats.
A3	A named contact and an agreed reply time for disputes.	A disputed status stays stuck and the SME waits.
A4	A daily export of statuses.	We cannot compare our records with the bank's.
A5	Test applications, marked as tests, before the first real one.	The first real customer becomes the test.
A6	Notice before the API changes.	A change can break our checks without anyone seeing.

Closing the call

"So we are clear on our side: we confirm every answer with a second read and your written confirmation before we tell the customer. We show 'accepted in principle' until you give final approval. And we start with a small pilot where our analyst checks each status with you. I will send this in an email today, so we can agree it in writing."

Task 5: Automating the Request

Today the customer gathers and uploads 12 months of files by hand. The plan replaces the two uploads in order of value, keeps a person wherever a mistake would reach the bank, and measures the result on both sides.

5.1 What to automate first, and why

The bank statement first. It is the evidence, and today it is our weakest file: scanned pages, edited balances, twelve downloads by hand. A feed straight from the bank removes the download and the risk of tampering. It works through open banking: the customer lets a provider licensed by SAMA (the Saudi Central Bank) read their bank data, with their consent. It is also cheaper per customer: one provider covers many banks, while invoices need an integration for each accounting tool.

OPTION	WHAT IT REMOVES	WHAT IT NEEDS	ORDER
Bank statement feed	The customer's twelve PDF downloads, and our unreadable-page and edited-balance risks.	A SAMA-licensed open-banking provider and the customer's consent. One provider covers many banks.	**First**
Invoice feed from accounting software	The Excel export, and re-typed or edited invoice files.	One integration per accounting tool, so a long list. Start with the one or two most used.	**Second**
Better manual upload	Back-and-forth over wrong or incomplete files.	A checklist, checks at upload, and reminders. Nothing to wait for from a partner.	**Alongside, and the fallback**

Caution. We do not yet know which banks are covered or how much history the feed returns, so manual upload stays as the fallback and we measure how many customers use the feed.

5.2 Where the data could come from instead of uploads

The customer connects, we do not scrape. In March 2026 SAMA licensed two open-banking providers for account information. ZATCA, the tax authority, checks and stamps business-to-business invoices electronically before they reach the buyer, so a VAT-registered wholesaler's invoices already exist in structured form somewhere. Both need checking before we rely on them.

DATA	COMES FROM	THE CUSTOMER	TO CHECK
Bank transactions	A SAMA-licensed open-banking provider, using its account-information service.	Picks the bank, signs in at the bank, and agrees to share 12 months.	Which banks are covered, and how many months of history come back.
Invoices	The accounting or e-invoicing software the SME already uses, through its API.	Connects the account and agrees to share.	That the SME uses one (Task 1, question Q1).

DATA	COMES FROM	THE CUSTOMER	TO CHECK
Cross-checks, later	VAT returns and commercial registration.	Agrees to share.	Whether a route exists.

5.3 Where a human stays in the loop

Automate what is the same every time. Keep a person where a wrong answer would reach the bank, or where the choice belongs to someone else.

STEP	DONE BY	WHY
Giving and withdrawing consent	Customer	It is their data. Each connection has a stated purpose, an end date and a way to withdraw.
Confirming the connected account is theirs	Customer	A wrong account gives a wrong package.
Reading, matching and flagging	System	The same rules every time. Every figure opens to its rows.
Matches made with high confidence	System	Spot-checked each month. On the sample data every one was right, but that is only sample data.
Low-confidence matches and blocking flags	Analyst	A wrong guess here would reach the bank.
Sending to the bank	Analyst, at first	Until the measured error rate is low. After that the analyst sees only the exceptions.
The credit decision	Bank	Not ours to make.

5.4 How we'd know it's better

FOR	WE WANT TO SHOW	MEASURED BY
Customer	It is faster and easier.	Time from starting to a submitted package. Share of customers who finish. Rounds of "needs changes". One question at the end: "How hard was this?"
Bank	Packages are easier to review and trust.	Share of packages returned with data questions. Time from receipt to decision. Share of our flags that the bank agrees were real.
Both	It is no less accurate.	Each month, check a sample of automated packages against the source files. Wrong figures must be at or below the manual flow.

Caution. We will not have enough volume for a proper A/B test (two groups of customers, one on each flow), so we compare before and after, and we keep the monthly audit. If the audit finds more wrong figures than the manual flow, we pause the feed for that source.

AI Tools Used

Claude (Anthropic). Helped write and review the prototype code, the tests and the sample-data generator, and helped draft and check the report text and diagrams. I made the scope and design calls, ran the prototype, and checked its numbers against the answer key.

Copilot. Produced a first set of sample data. I rejected it because of repeated transaction ids, payments bunched on a few days, and unreadable Arabic in the PDF. After adjusting the data and the Python code that creates the data files, we arrived at the final version of the data.